```
Error cuadr\'atico:
2249.94
```

B.3. Sistemas de ecuaciones

B.3.1. *LU*

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-

# ----------------------------------------------------------------------
# Compendio de algoritmos.
# Matemáticas para Ingeniería. Métodos numéricos con Python.
# Copyright (C) 2017  Los autores del texto.
# This program is free software: you can redistribute it and/or modify
# it under the terms of the GNU General Public License as published by
# the Free Software Foundation, either version 3 of the License, or
# (at your option) any later version.
# This program is distributed in the hope that it will be useful,
# but WITHOUT ANY WARRANTY; without even the implied warranty of
# MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
# GNU General Public License for more details.
# You should have received a copy of the GNU General Public License
# along with this program.  If not, see <http://www.gnu.org/licenses/>
# ----------------------------------------------------------------------

# Implementación del método LU y algunos casos de salida.

from math import *
from pprint import pprint


def lu(A):
    n = len(A)
    # crear matrices nulas
    L = [[0.0 for x in range(n)] for x in range(n)]
    U = [[0.0 for x in range(n)] for x in range(n)]

    # Doolittle
    L[0][0] = 1.0
    U[0][0] = float(A[0][0])

    if abs(L[0][0]*U[0][0]) <= 1e-15:
        print "Imposible descomponer"
        return
```

```
39
40      for j in range(1, n):
41          U[0][j] = A[0][j]/L[0][0]
42          L[j][0] = A[j][0]/U[0][0]
43
44      for i in range(1, n-1):
45          L[i][i] = 1.0
46          s = sum([L[i][k]*U[k][i] for k in range(i)])
47          U[i][i] = float(A[i][i])-float(s)
48
49          if abs(L[i][i]*U[i][i]) <= 1e-15:
50              print "Imposible descomponer"
51              return
52
53          for j in range(i+1, n):
54              s1 = sum([L[i][k]*U[k][j] for k in range(i)])
55              s2 = sum([L[j][k]*U[k][i] for k in range(i)])
56              U[i][j] = float(A[i][j])-float(s1)
57              L[j][i] = (float(A[j][i])-float(s2))/float(U[i][i])
58
59      L[n-1][n-1] = 1.0
60      s3 = sum([L[n-1][k]*U[k][n-1] for k in range(n)])
61      U[n-1][n-1] = float(A[n-1][n-1])-float(s3)
62
63      if abs(L[n-1][n-1]*U[n-1][n-1]) <= 1e-15:
64          print "Imposible descomponer"
65          return
66
67      print "Matriz L:\n"
68      pprint(L)
69      print
70      print "Matriz U:\n"
71      pprint(U)
72
73
74  A = [[4, 3], [6, 3]]
75  print "Matriz A:\n"
76  pprint(A)
77  print
78  lu(A)
79  print "---------"
80
81  A = [[0, 1], [1, 1]]
82  print "Matriz A:\n"
83  pprint(A)
84  print
85  lu(A)
86  print "---------"
87
```

```
88 A = [[3, 1, 6], [-6, 0, -16], [0, 8, -17]]
89 print "Matriz A:\n"
90 pprint(A)
91 print
92 lu(A)
93 print "---------"
94
95 A = [[1, 2, 3], [2, 4, 5], [1, 3, 4]]
96 print "Matriz A:\n"
97 pprint(A)
98 print
99 lu(A)
100 print "---------"
```

Salida

```
Matriz A:

[[4, 3], [6, 3]]

Matriz L:

[[1.0, 0.0], [1.5, 1.0]]

Matriz U:

[[4.0, 3.0], [0.0, -1.5]]
---------
Matriz A:

[[0, 1], [1, 1]]

Imposible descomponer
---------
Matriz A:

[[3, 1, 6], [-6, 0, -16], [0, 8, -17]]

Matriz L:

[[1.0, 0.0, 0.0], [-2.0, 1.0, 0.0], [0.0, 4.0, 1.0]]

Matriz U:

[[3.0, 1.0, 6.0], [0.0, 2.0, -4.0], [0.0, 0.0, -1.0]]
---------
Matriz A:

[[1, 2, 3], [2, 4, 5], [1, 3, 4]]
```

```
Imposible descomponer
---------
```

B.3.2. Jacobi

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-

# ------------------------------------------------------------------------
# Compendio de algoritmos.
# Matemáticas para Ingeniería. Métodos numéricos con Python.
# Copyright (C) 2017  Los autores del texto.
# This program is free software: you can redistribute it and/or modify
# it under the terms of the GNU General Public License as published by
# the Free Software Foundation, either version 3 of the License, or
# (at your option) any later version.
# This program is distributed in the hope that it will be useful,
# but WITHOUT ANY WARRANTY; without even the implied warranty of
# MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
# GNU General Public License for more details.
# You should have received a copy of the GNU General Public License
# along with this program.  If not, see <http://www.gnu.org/licenses/>
# ------------------------------------------------------------------------

# Implementación del método de Jacobi y algunos casos de salida.

from math import *
from pprint import pprint


def distinf(x, y):
    return max([abs(x[i]-y[i]) for i in range(len(x))])


def Jacobi(A, b, x0, TOL, MAX):
    n = len(A)
    x = [0.0 for x in range(n)]
    k = 1
    while k <= MAX:
        for i in range(n):
            if abs(A[i][i]) <= 1e-15:
                print "Imposible iterar"
                return
            s = sum([A[i][j]*x0[j] for j in range(n) if j != i])
            x[i] = (b[i]-float(s))/float(A[i][i])
        pprint(x)
        if distinf(x, x0) < TOL:
            print r"Soluci\'on encontrada"
```

```
            return
        k += 1
        for i in range(n):
            x0[i] = x[i]
    print "Iteraciones agotadas"
    return

A = [[2, 1], [5, 7]]
b = [11, 13]
x0 = [1, 1]
print "Matriz A:\n"
pprint(A)
print
print "Vector b:\n"
pprint(b)
print
print "Semilla x0:\n"
pprint(x0)
print "\n"+r"Iteraci\'on de Jacobi"
# TOL = 10^-5, MAX = 50
Jacobi(A, b, x0, 1e-5, 50)


A = [[10, -1, 2], [-1, 11, -1], [2, -1, 10]]
b = [6, 25, -11]
x0 = [0, 0, 0]
print "\n"+"Matriz A:\n"
pprint(A)
print
print "Vector b:\n"
pprint(b)
print
print "Semilla x0:\n"
pprint(x0)
print "\n"+r"Iteraci\'on de Jacobi"
# TOL = 10^-10, MAX = 50
Jacobi(A, b, x0, 1e-10, 50)
```

Salida

```
Matriz A:

[[2, 1], [5, 7]]

Vector b:

[11, 13]

Semilla x0:
```

```
[1, 1]

Iteraci\'on de Jacobi
[5.0, 1.1428571428571428]
[4.928571428571429, -1.7142857142857142]
[6.357142857142857, -1.6632653061224494]
[6.331632653061225, -2.683673469387755]
[6.841836734693878, -2.6654518950437316]
[6.832725947521865, -3.0298833819241984]
[7.014941690962099, -3.0233756768013325]
[7.0116878384006665, -3.1535297792586428]
[7.076764889629321, -3.151205598857619]
[7.075602799428809, -3.197689206878086]
[7.098844603439043, -3.1968591424491493]
[7.098429571224575, -3.213460431027888]
[7.106730215513944, -3.2131639794461244]
[7.106581989723062, -3.2190930110813887]
[7.109546505540695, -3.2189871355164734]
[7.1094935677582365, -3.221104646814782]
[7.110552323407391, -3.2210668341130266]
[7.110533417056513, -3.221823088148137]
[7.110911544074068, -3.221809583611795]
[7.110904791805897, -3.22207967433862]
[7.11103983716931, -3.2220748512899258]
[7.111037425644962, -3.2221713122637925]
[7.1110856561318965, -3.2221695897464024]
[7.111084794873201, -3.222204040094212]
[7.111102020047106, -3.22220342490943]
[7.111101712454715, -3.2222157286050757]
[7.111107864302538, -3.2222155088962245]
Soluci\'on encontrada

Matriz A:

[[10, -1, 2], [-1, 11, -1], [2, -1, 10]]

Vector b:

[6, 25, -11]

Semilla x0:

[0, 0, 0]

Iteraci\'on de Jacobi
[0.6, 2.272727272727273, -1.1]
[1.0472727272727274, 2.227272727272727, -0.9927272727272728]
[1.0212727272727273, 2.277685950413223, -1.0867272727272728]
[1.0451140495867768, 2.266776859504132, -1.0764859504132231]
```

```
[1.0419748760330578, 2.2698752817430505, -1.0823451239669422]
[1.0434565529676934, 2.2690572501878283, -1.0814074470323065]
[1.043187214425244, 2.2692771914486713, -1.0817855855747558]
[1.0432848362598182, 2.269218329895499, -1.0817097237401818]
[1.0432637777375864, 2.269234101138149, -1.0817351342624137]
[1.0432704369662977, 2.269229876679561, -1.0817293454337025]
[1.0432688567546966, 2.269231008321145, -1.0817310997253036]
[1.0432693207771753, 2.26923070518449, -1.0817306705188248]
[1.043269204622214, 2.269230786387123, -1.0817307936369862]
[1.0432692373661094, 2.2692307646350205, -1.0817307622857304]
[1.0432692289206482, 2.2692307704618524, -1.08173077100972]
[1.043269231248129, 2.2692307689009934, -1.0817307687379443]
[1.0432692306376883, 2.269230769319108, -1.0817307693595264]
[1.043269230803816, 2.269230769207106, -1.081730769195627]
[1.0432692307598361, 2.269230769237108, -1.0817307692400526]
Soluci\'on encontrada
```

B.3.3. Gauss-Seidel

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-

# ------------------------------------------------------------------------
# Compendio de algoritmos.
# Matemáticas para Ingeniería. Métodos numéricos con Python.
# Copyright (C) 2017  Los autores del texto.
# This program is free software: you can redistribute it and/or modify
# it under the terms of the GNU General Public License as published by
# the Free Software Foundation, either version 3 of the License, or
# (at your option) any later version.
# This program is distributed in the hope that it will be useful,
# but WITHOUT ANY WARRANTY; without even the implied warranty of
# MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
# GNU General Public License for more details.
# You should have received a copy of the GNU General Public License
# along with this program.  If not, see <http://www.gnu.org/licenses/>
# ------------------------------------------------------------------------

# Implementación del método de Gauss-Seidel y algunos casos de salida.

from math import *
from pprint import pprint


def distinf(x, y):
    return max([abs(x[i]-y[i]) for i in range(len(x))])

```